

REVERTPAY V1

TOP INTERVIEW QUESTIONS & ANSWERS

MAGNAQUEST DIRECT TECHNICAL ROUND

SIVA CHARAN

What this is: the 50 questions most likely to decide your round — extracted from the 69-page Master Handbook, every answer mapped to the source-verified implementation. No filler, no re-teaching of Java/SQL/DSA.

Source of truth: RevertPay v1 source tree (Spring Boot 4.1.1 · Java 25 · PostgreSQL/Neon · React 19) as verified in the Master Handbook.

Label	Meaning
[CODE]	directly confirmed from the implementation
[INFERENCE]	technical conclusion based on the implementation / framework
[GAP]	not implemented / incomplete — answer honestly, then give the fix
[GENERAL]	general technical explanation

The honesty pattern to memorize: “That isn't implemented in my V1. In the current version X happens instead. I would implement it by...” — every [GAP] answer below uses it.

Contents: 9 sections · 50 full-format Q&A · Top-15 must-know list · 5-minute revision page · 10 dangerous follow-ups · 60-second project explanation.

SECTION 1 — PROJECT UNDERSTANDING

Q1. "So tell me about RevertPay — what is it and what problem does it solve?"

Priority: MUST KNOW

What interviewer is testing: Whether you can explain your own project crisply, and whether you understand the domain (escrow) or just wrote CRUD code.

Ideal natural answer: "RevertPay is an escrow-based payment platform I built with Spring Boot and PostgreSQL. The core idea: a buyer's money doesn't go straight to the seller — it goes into a system-controlled holding account called ESCROW_HOLDING. The seller ships only after money is locked in escrow, and the buyer's delivery confirmation triggers a ledger transaction that releases the money to the seller. So neither party has to trust the other — they only have to trust the platform. Every rupee movement is recorded as a double-entry ledger transaction, and every escrow moves through a validated state machine with a full audit log. Auth is JWT-based with Spring Security."

Deep explanation: The problem being solved is counterparty trust risk in online trades: buyers fear paying and receiving nothing; sellers fear shipping and not being paid. Escrow removes trust between the two parties by inserting a neutral holder (the platform's SYSTEM account) between commitment and settlement. What makes your implementation credible is that the "holding" is not a claimed number — it is provable from the ledger, and the lifecycle is enforced by a centralized state machine, not by discipline.

Follow-up: "Why would anyone trust your platform to hold the money?"

Follow-up answer: "Fair challenge — trust in a real escrow comes from regulation, audits and licensing, which a v1 can't have. What I can show is engineering honesty: the escrow holdings are derivable from an append-only ledger with a conservation law — total held equals the sum of active escrows, and one SQL query proves it. In production you'd add regulated custody, external audits, and compliance on top of exactly this kind of ledger core."

Trap: Calling it "a payment app like PayPal". That invites PayPal-scale questions (dispute networks, chargebacks, card rails) you can't defend. Anchor everything to escrow and the ledger.

Project evidence: `AccountService.createEscrowHoldingAccount()` creates the SYSTEM-owned pot at startup; `EscrowService` orchestrates the lifecycle; `LedgerService.transfer()` is the only money path. [CODE]

Q2. "Explain the complete architecture of your project."

Priority: MUST KNOW

What interviewer is testing: Whether you know your actual layers and class names — not a generic three-tier diagram.

Ideal natural answer: "It's a two-module repo. The frontend is a React 19 + Vite SPA that stores a JWT in localStorage and calls the API with fetch — in dev, Vite proxies /api to Tomcat on 8080. On the backend: every request passes through my `JwtAuthenticationFilter`, which verifies the token's signature and expiry, loads the user from the database, and sets a `ROLE_BUYER` / `ROLE_SELLER` authority into the `SecurityContext`. Then a stateless `SecurityFilterChain` allows only register and login publicly and demands authentication for everything else. Controllers like `EscrowController` stay thin — they just extract the Bearer token and delegate. Input arrives as DTOs validated with annotations like `@Positive` on amount. Services hold all the business rules — `EscrowService`, `LedgerService`, `EscrowTransitionService`. Repositories are Spring Data interfaces with derived queries. Hibernate generates the schema with `ddl-auto=update` against PostgreSQL running on Neon."

Deep explanation: The flow is: React → HTTP (Bearer JWT) → `JwtAuthenticationFilter` → `SecurityFilterChain` authorization → Controller → DTO validation → Service (business logic + `@Transactional` boundaries) → Repository (Spring Data) → JPA/Hibernate → PostgreSQL. The scheduler (`@Scheduled(fixedRate=5000)`) also enters through the service layer, which proves the layering pays off — expiry reuses the exact same rules as user actions.

Follow-up: "Why are business rules in the service layer and not the controller?"

Follow-up answer: "Three reasons: testability — I can unit-test `EscrowTransitionService` with no Spring context; transaction boundaries — `@Transactional` belongs where multiple repository calls compose into one business operation; and reuse — my scheduler calls the same `checkExpiry` logic users trigger, so rules can't diverge between paths."

Trap: Reciting "controller-service-repository" without a single real class name. Interviewers probe immediately: "show me where X lives" — if you can't name `EscrowTransitionService` or `JwtAuthenticationFilter`, you sound like you followed a tutorial.

Project evidence: `EscrowController`, `EscrowService`, `EscrowRepository`, `JwtAuthenticationFilter`, `SecurityConfig`, `application.properties` (Neon URL, `ddl-auto=update`). [CODE]

Q3. "Walk me through ONE complete escrow transaction — from initiate to money reaching the seller."

Priority: MUST KNOW

What interviewer is testing: End-to-end fluency: states, endpoints, ownership checks, and where money actually moves.

Ideal natural answer: "Take buyer ACC5 and seller ACC9 trading an iPhone for ₹600. First, the buyer calls `POST /api/escrow/initiate` with the seller's account number, item and amount, plus an `Idempotency-Key` header. The service resolves the seller from that account, checks the caller is a BUYER and the target is a SELLER, and that they're different people. It saves the escrow in `PAYMENT_INITIATED` with a confirmation deadline 40 seconds out. No money moves yet. The buyer then confirms — state `CREATED` — and funds it: a double-entry transfer debits ACC5 ₹600 and credits `ESCROW_HOLDING` ₹600, state `FUNDED`. The seller ships — `SHIPPED`, and we store `shippedAt` plus an `autoReleaseAt` five minutes out. Finally the buyer confirms delivery: another double-entry transfer, this time `ESCROW_HOLDING` to ACC9, and the state becomes `RELEASED`. Every step appends a row to `escrow_status_log` with who did it and why."

Deep explanation: Key beats the interviewer listens for: (1) money moves at exactly two points — fund and release; (2) identity always comes from the JWT, never the request body; (3) the ledger pair and the status change commit atomically in fund and release (both are `@Transactional`); (4) the audit log records actor + reason per transition.

Follow-up: "Where exactly is the ₹600 between funding and release?"

Follow-up answer: "In the `ESCROW_HOLDING` system account. Its derived balance — sum of credits minus debits — reflects every in-flight escrow, and because each ledger row carries `reference_id` = the escrow id, I can prove per-trade exactly what's held."

Trap: Saying money moves at initiate. It doesn't — `PAYMENT_INITIATED` is a zero-money review state. That single error tells the interviewer you don't actually know your own flow.

Project evidence: `EscrowService.transactionRequest / confirmTransaction / fundTransaction / shipTransaction / confirmEscrow`; `LedgerService.transfer`. [CODE]

Q4. "Explain your project in 30 seconds. Now in 1 minute."

Priority: MUST KNOW

What interviewer is testing: Communication discipline — can you compress without losing the essence?

Ideal natural answer (30s): "RevertPay is an escrow payment app: buyer's money goes into a system holding account, the seller ships, and delivery confirmation releases the money via a double-entry ledger. Every escrow is a state machine — initiated, confirmed, funded, shipped, released — with illegal transitions rejected and every change audit-logged. Spring Boot, PostgreSQL, JWT auth, React frontend."

Ideal natural answer (1 min): Add: "The part I'm proudest of is the money-safety model. There's no balance column — balances are always derived by summing an append-only ledger, so there's a complete audit trail and no lost-update problem. Each transfer writes one debit and one credit in a single database transaction. Escrow updates are guarded by JPA optimistic locking, so two concurrent releases can't double-pay — only one commit survives and the loser's ledger rows roll back with its transaction. I also know v1's honest limits: refunds don't move money yet, idempotent replay isn't wired, and balance checks before funding are missing — I can explain exactly how I'd add each."

Deep explanation: The 1-minute version earns trust because it volunteers limitations — that's the strongest senior signal available. Interviewers reflexively distrust flawless projects.

Follow-up: "What's the single most impressive engineering decision in it?"

Follow-up answer: "Derived balances from an append-only ledger, combined with version-guarded release — together they make 'money appearing or vanishing' and 'double-paying a seller' structurally impossible, not just carefully avoided."

Trap: A feature list ("I used Spring Boot, JWT, React, Hibernate...") with no story. Lead with the problem and the money path; tech names come second.

Project evidence: Whole codebase; limitation list verified in source (no refund flow, no replay handling, no balance check). [CODE] + [GAP]

Q5. "Who are the actors in the system, and what can each one do?"

Priority: HIGH

What interviewer is testing: RBAC clarity — and whether you'll overclaim an admin panel that doesn't exist.

Ideal natural answer: "Three roles in the enum: BUYER, SELLER, ADMIN. Buyers initiate escrows, confirm or decline them inside 40 seconds, fund them, confirm delivery — which releases money — and raise disputes. Sellers see funded escrows and mark them shipped. ADMIN... exists in the role enum and in the registration dropdown, but honestly, v1 has zero admin-only endpoints — no dispute resolution UI, no refund action. That's a known gap: the state machine models UNDER_REVIEW to RELEASED and REFUNDED, but nothing invokes them yet."

Deep explanation: The role is stored on the user row, and the JWT filter builds a ROLE_X authority from the database value on every request — so authorization uses fresh role data, not a stale token claim. Endpoint-level, only register/login are public; everything else needs authentication. Role-specific rules (buyer-only funding, seller-only shipping) are enforced inside the services.

Follow-up: "How would you add real admin capabilities?"

Follow-up answer: "Server-side role provisioning only — stop trusting the client-chosen role at registration; gate an admin resolution endpoint with @PreAuthorize('hasRole(ADMIN)'); implement UNDER_REVIEW resolution: release or refund, each a transactional money movement with the admin's id as the audit actor."

Trap: Claiming "admins resolve disputes through a panel". There is no admin endpoint anywhere in the code. Volunteering the gap scores; getting caught in it kills you.

Project evidence: Role enum; AuthService.register accepts client-supplied role (itself a gap to acknowledge); DisputeService.Underreview sets OPENED but no resolution path exists. [CODE] + [GAP]

SECTION 2 — ESCROW + STATE MACHINE

Q6. "Why escrow at all? Why not just a simple wallet-to-wallet transfer?"

Priority: MUST KNOW

What interviewer is testing: Domain reasoning — did you solve a problem or just wire endpoints together?

Ideal natural answer: "Because payment risk is temporal. In an online trade, the buyer must pay before the item arrives — a plain transfer has no undo once committed. Escrow inserts a hold state between commitment and settlement: the buyer commits at fund, the seller ships against locked money, and settlement only happens on delivery confirmation — or the money can flow back through dispute resolution. That 'gap' is exactly what my FUNDED and SHIPPED states model, and because the hold lives in a real ledger account, what's held is provable, not just claimed."

Deep explanation: A transfer is one irreversible event; escrow is a lifecycle. That's why the state machine exists at all — states are how you give an irreversible rail a reversible business experience. The name RevertPay reflects the reversal path (refund/review states), even though the refund money-flow is future work.

Follow-up: "Which real-world products work this way?"

Follow-up answer: "Marketplace escrows — e-commerce escrow services, freelancing platforms that hold milestone payments, and card-chargeback systems are the consumer-facing cousins. The ledger + hold-account pattern is how real payment cores model it."

Trap: "Escrow is more secure." Vague. Secure how? Name the risk (temporal counterparty risk) and the mechanism (hold state + provable holdings).

Project evidence: `EscrowStatus` lifecycle; `ESCROW_HOLDING` account; ledger `reference_id` ties held money to a specific escrow. [CODE]

Q7. "Walk me through your state machine — every state and every legal transition. Verbally draw it."

Priority: MUST KNOW

What interviewer is testing: Whether you can reproduce the exact transition table from `EscrowTransitionService.isValid()` — memory of your own core logic.

Ideal natural answer: "Nine states. `PAYMENT_INITIATED` can go three ways: buyer confirms to `CREATED`, buyer cancels to `CANCELLED`, or the scheduler expires it to `EXPIRED` after the 40-second deadline. `CREATED` has exactly one exit: buyer funds it to `FUNDED` — that's the first money movement. `FUNDED` has one exit: seller ships to `SHIPPED`, which stamps `shippedAt` and an `autoReleaseAt` five minutes out. `SHIPPED` branches two ways: buyer confirms delivery to `RELEASED` — money leaves escrow — or buyer raises a dispute to `UNDER_REVIEW`. `UNDER_REVIEW` can end in `RELEASED` or `REFUNDED`. And four states are terminal — `CANCELLED`, `EXPIRED`, `RELEASED`, `REFUNDED` — the `isValid` switch returns false for any exit. Every legal transition also appends an `escrow_status_log` row with from, to, actor and reason."

Deep explanation: Say it as a story with exactly one branching point after money moved (`SHIPPED`) — interviewers check whether your mental model matches the code's shape. Money moves on exactly two edges: `CREATED→FUNDED` and `SHIPPED→RELEASED`.

Follow-up: "Which transitions move money?"

Follow-up answer: "Only two: funding (buyer → `ESCROW_HOLDING`) and release (`ESCROW_HOLDING` → seller). Every other transition is pure state — that separation is deliberate, so cleanup states like `EXPIRED` never touch money."

Trap: Inventing transitions that feel natural but don't exist — `FUNDED→CANCELLED`, or expiry from `CREATED`. Expiry is only reachable from `PAYMENT_INITIATED`. That precision is what separates owners from tourists.

Project evidence: `EscrowTransitionService.isValid()` — exhaustive switch over all enum constants; terminal states return false. [CODE]

Q8. "What is PAYMENT_INITIATED and why does it exist as a separate state before funding?"

Priority: HIGH

What interviewer is testing: Whether you understand why the flow has a review gate.

Ideal natural answer: "It's a zero-money review window. After initiate, nothing has been paid — the buyer gets one final look at seller, item and amount, and must explicitly confirm within 40 seconds, turning it into CREATED, before any funding is possible. Decline cancels it; silence expires it. Because no money has moved, all three outcomes are trivially safe — no reversing entries, no compensation. It's deliberately cheap."

Deep explanation: Two decisions are separated on purpose: intent ("yes, I'm paying this exact escrow") and money commitment (funding). Merging them would mean every accidental initiate needs a refund path. The review state converts user-error cleanup into simple state expiry.

Follow-up: "Why 40 seconds — that seems arbitrary?"

Follow-up answer: "It is — it's a demo-tunable value chosen so expiry is observable in a live demo alongside the 5-second scheduler sweep. In production it'd be minutes or hours and come from configuration per marketplace."

Trap: Confusing PAYMENT_INITIATED with FUNDED, or saying "payment is pending processing" — there's no payment processing at this stage, full stop.

Project evidence: `transactionRequest` sets `confirmationDeadline = now.plusSeconds(40)` and status `PAYMENT_INITIATED`. [CODE]

Q9. "Explain the confirmation deadline and expiry mechanics end to end."

Priority: MUST KNOW

What interviewer is testing: Scheduler design: how deadlines are stored, swept, and survive restarts.

Ideal natural answer: "At initiate I store a `confirmationDeadline` of now plus 40 seconds on the escrow row — it's persisted in PostgreSQL, not in memory. A Spring scheduler runs every five seconds with `fixedRate=5000`, loads all escrows still in `PAYMENT_INITIATED`, and calls `checkExpiry` for each. If now is past the deadline, `checkExpiry` drives the same centralized transition everyone else uses — to `EXPIRED`, with actor id null to mark it as a system action and reason 'Expired Session'. Because the deadline lives in the database, a restart is harmless — after downtime, the next tick expires everything overdue in one pass."

Deep explanation: Design points to name: time-based (not timer-per-entity) expiry; idempotent sweep (re-expiring is impossible — terminal state); the system writes through the same transition service as users, so audit logging stays uniform; `actorId=null` distinguishes system actions in the audit trail.

Follow-up: "What if the application is down for ten minutes?"

Follow-up answer: "Nothing degrades — deadlines are timestamps in the DB. On restart, the first sweep expires every escrow whose deadline passed during the outage. No catch-up logic needed because expiry is computed from stored time, not from counting ticks."

Trap: Saying "each escrow has its own timer". It doesn't — there's one periodic sweep and a stored deadline. Also don't claim the scheduler "locks" rows; protection comes from `@Version`.

Project evidence: `EscrowScheduler (@Scheduled(fixedRate=5000))`, `EscrowService.checkExpiry`, `confirmationDeadline`. [CODE]

Q10. "How does funding work — endpoint, checks, money movement, state change?"

Priority: MUST KNOW

What interviewer is testing: Whether you know your most security-critical method line by line.

Ideal natural answer: "POST `/api/escrow/{id}/fund`. The controller extracts the Bearer token; the service method is `@Transactional`. It re-validates the token, loads the escrow, and checks the token's user id equals the escrow's buyer id — so only the buyer of that escrow can fund it. Then it resolves two accounts: ACC plus the buyer's id, and `ESCROW_HOLDING`. It calls `ledgerService.transfer` with the escrow's amount and the escrow id as reference —

one debit on the buyer, one credit on escrow, both inside the same database transaction. Then the transition service moves status CREATED to FUNDED and an audit row is written. If any step throws — bad state, bad token, missing account — the whole transaction rolls back: no ledger rows, no status change."

Deep explanation: The headline is atomicity: two ledger inserts + escrow status update + audit log = one commit or nothing. The ownership check (token id vs escrow.buyer.id) is object-level authorization — the reason user A can't fund with user B's escrow even with a valid token.

Follow-up: "What if the buyer double-clicks and funds twice?"

Follow-up answer: "The second request hits the state machine: FUNDED to FUNDED is illegal, so it throws `InvalidTransitionException` — no second transfer. In v1 that surfaces as an ugly 500 instead of a friendly 'already funded'; mapping it to a 409 is on my fix list. Money-wise it's safe."

Trap: Forgetting the ownership check exists here, or claiming the transfer "updates balances" — it appends two ledger rows; balances are derived later.

Project evidence: `EscrowService.fundTransaction` — `@Transactional`, buyer check, `ACC{id}` + `ESCROW_HOLDING` lookups, transfer, transition. [CODE]

Q11. "How do shipping and delivery confirmation work?"

Priority: HIGH

What interviewer is testing: The seller-side flow, the auto-release field — and whether you'll overclaim auto-release.

Ideal natural answer: "Shipping is seller-only: the service checks the token belongs to the escrow's seller, then transitions FUNDED to SHIPPED and stamps `shippedAt` and `autoReleaseAt` — now plus five minutes. Delivery confirmation is buyer-only: it moves the money — `ESCROW_HOLDING` to the seller's account via a double-entry transfer — and transitions SHIPPED to RELEASED, all in one `@Transactional` method. One honest point: `autoReleaseAt` is stored and shown in the UI, but the job that would enforce it is an empty stub in v1 — nothing auto-releases today. The field is future-compatible schema, not a working feature."

Deep explanation: Note the symmetry of the two ownership checks: ship compares against `getSeller().getId()`, confirm-delivery against `getBuyer().getId()`. And note that release is the only path where money exits escrow in v1.

Follow-up: "How would you implement auto-release properly?"

Follow-up answer: "A scheduled sweep over SHIPPED escrows where `auto_release_at` is past — indexed, batched with `SKIP LOCKED` so parallel workers don't double-claim — and per escrow: the same transactional release the buyer endpoint uses, with the system as audit actor, plus `ShedLock` so only one instance sweeps in a cluster."

Trap: "The system auto-releases after five minutes." It does NOT — saying this and being asked "show me the job" is fatal. Volunteer the stub.

Project evidence: `shipTransaction` (sets both timestamps), `confirmEscrow` (`@Transactional` release), `EscrowAutoReleaseJob` = empty class. [CODE] + [GAP]

Q12. "What happens on an illegal transition — and why did you centralize transition validation?"

Priority: MUST KNOW

What interviewer is testing: Design judgment: one validation choke point vs scattered ifs; error semantics.

Ideal natural answer: "All state changes funnel through one service: `EscrowTransitionService.transition()`. It reads the current status, asks `isValid` — an exhaustive switch — and if the move is illegal it throws `InvalidTransitionException` naming both states. Nobody can bypass it: not the buyer endpoints, not the seller, not the scheduler — they all call the same method, and each success also writes an audit row. Two things I'd flag honestly: today that exception isn't mapped by any handler, so the client sees a 500 where a 409 Conflict is semantically right; and the exhaustive switch is a nice compiler trick — if I add a state to the enum, the code won't compile until I define its legal edges."

Deep explanation: Centralization gives: one legality source (no drift between endpoints), automatic audit coverage, and a single place to evolve rules (e.g., adding an approval state). The 500-vs-409 distinction is a classic probe — 409 means "valid request, conflicts with the resource's current state", which is exactly an illegal

transition.

Follow-up: "Why not enforce transitions with DB triggers instead?"

Follow-up answer: "Business legality belongs in the domain layer where it's testable and readable; the database's job in this design is concurrency truth via the version column. Triggers would duplicate the same table in SQL with worse testability and surprise-at-a-distance debugging."

Trap: "Each endpoint validates its own status checks." That was actually v1's earlier design — the hardcoded checks are commented out in the code after I moved rules into the transition service. That refactor story is a strong answer; use it.

Project evidence: `EscrowTransitionService.transition/isValid`; commented-out per-endpoint status checks in `EscrowService`; empty `GlobalExceptionHandler`. [CODE]

SECTION 3 — DOUBLE-ENTRY LEDGER

Q13. "What is double-entry accounting, and why did you use it in RevertPay?"

Priority: MUST KNOW

What interviewer is testing: The single most important design in your project — depth here signals real payments thinking.

Ideal natural answer: "Double-entry means every money movement is recorded as two equal, opposite sides: a DEBIT on one account and a CREDIT of the same amount on another, written together in one database transaction. That gives me a conservation law — total debits always equal total credits — so money structurally cannot appear or vanish without a trace. I chose it over a simple balance column because payments demand an audit trail: every row says which account, which direction, how much, when, and — via `reference_id` — which escrow caused it. It's the same model real payment cores use."

Deep explanation: The three payoffs: (1) auditability — an append-only history you can replay; (2) reconciliation — the $\sum \text{debits} = \sum \text{credits}$ invariant is checkable with one query, and 'escrow holdings' can be recomputed independently of business tables; (3) concurrency-friendliness — balances are never read-modify-written, entries only append, so transfers don't fight over rows.

Follow-up: "What's the invariant, exactly — and how would you check it?"

Follow-up answer: "Sum of all DEBIT amounts equals sum of all CREDIT amounts, always — my transfer method only writes pairs, atomically. A nightly job runs one SELECT with two filtered SUMs and alerts if they diverge by even a paisa."

Trap: Reciting "debit means minus, credit means plus" as if absolute. Convention-dependent — my code treats balance as credits minus debits. The equality of the two sides is the invariant; the sign convention is just presentation.

Project evidence: `LedgerService.doDebit/doCredit/transfer`; `LedgerEntry` (account, Type, amount, `reference_id`). [CODE]

Q14. "Walk me through a ₹1000 escrow payment in the ledger — row by row."

Priority: MUST KNOW

What interviewer is testing: Whether you can trace actual money movement, not hand-wave "it records transactions".

Ideal natural answer: "Sure. New users get ₹5000 seeded, so buyer's ACC5 starts at 5000. Seed entries: DEBIT 5000 on ESCROW_HOLDING, CREDIT 5000 on ACC5 — the system 'mints' demo money, so the treasury account goes negative; that's expected and keeps conservation intact. Now the ₹1000 trade. Funding escrow 7: row one — DEBIT 1000, account ACC5, reference 7; row two — CREDIT 1000, account ESCROW_HOLDING, reference 7. ACC5's derived balance: 5000 minus 1000 = 4000. On delivery confirmation: DEBIT 1000 on ESCROW_HOLDING and CREDIT 1000 on ACC9, again reference 7. Seller now at 6000 with their seed. Escrow's net for trade 7 is back to zero — while the trade was in FUNDED or SHIPPED, that net was exactly 1000: the amount held."

Deep explanation: The running-balance story has three beats the interviewer checks: holdings are visible mid-flight (escrow net = amount held), settlement zeroes the trade's net, and every row carries the same `reference_id` so per-trade forensics are one WHERE clause.

Follow-up: "Your escrow account has a negative balance. Is that a bug?"

Follow-up answer: "No — it's the mint. Seeding user wallets debits the treasury; in double-entry the treasury absorbs the issuance, like a central bank's account going negative on issuance. The conservation law still holds globally; and since ESCROW_HOLDING also collects every funding credit, its net reflects exactly what's currently held for active trades."

Trap: Showing only the user-side entry and forgetting the pair — or panicking at the negative treasury balance instead of explaining it. Explaining the negative balance proactively is a senior signal.

Project evidence: `AccountService.createAccount` (₹5000 seed via transfer), `fundTransaction`, `confirmEscrow`. [CODE]

Q15. "Why not simply store a balance column and UPDATE it? Isn't deriving balances inefficient?"

Priority: MUST KNOW

What interviewer is testing: Whether the ledger design is a reasoned trade-off or accidental architecture.

Ideal natural answer: "A balance column creates two problems. Concurrency: updating a balance is read-modify-write — two simultaneous transfers can lose an update unless you lock or conditional-write. Integrity: a balance can drift from its history — one bug and you can't prove what the 'true' number is. With append-only entries, neither exists: inserts never conflict, and the balance is defined by the entries, so it can't disagree with them. The cost is real: computing a balance is $O(n)$ over the account's history, and that's the trade I consciously accepted in v1. When n grows, the standard fix is an incremental aggregate — a balance row updated inside the same transaction as the entry pair — so the ledger stays the source of truth and reads become $O(1)$."

Deep explanation: There's visible proof this was deliberate: `Account.java` contains a commented-out balance field — the design moved ledger-first. Efficiency of correctness beats efficiency of convenience in money systems; you optimize later without losing the audit property.

Follow-up: "So at what point would you add the balance column back?"

Follow-up answer: "When read latency or scan cost hurts — measured, not guessed. I'd add it as a cache, updated in the same transaction as the pair, reconciled nightly against the ledger. The invariant check stays; the column becomes an optimization, never the truth."

Trap: Either extreme: "balance columns are always wrong" (naive — every real bank materializes balances) or "deriving is fine forever" (ignores $O(n)$). Show you know both failure modes.

Project evidence: `Account.java` commented balance field; balance derived in `UserController.getMe` and `AccountController.findBalance`. [CODE]

Q16. "How are balances actually computed in your code right now?"

Priority: HIGH

What interviewer is testing: Precision — do you know your own implementation details, or just the concept?

Ideal natural answer: "In two places: `/api/me` and `/api/accounts/{accNo}/balance`. Both load the account, fetch all its `LedgerEntry` rows via `findByAccountId`, then loop in Java — every CREDIT adds to a running credit total, everything else adds to debits — and return credits minus debits. It's a plain `BigDecimal` accumulator. The frontend's My Account page displays that balance and even re-derives the same sums client-side from the returned ledger."

Deep explanation: Interviewers love this question because the precise answer differs from the assumed one — there's no SQL SUM, no query in the repository beyond `findByAccountId`; the summation is a Java for-loop. Knowing that detail — and its implications (full history loaded per read, N rows over the wire) — proves ownership.

Follow-up: "What's the complexity, and the first optimization you'd make?"

Follow-up answer: " $O(\text{history})$ per read, both in DB rows scanned and payload. Step one is an index on `ledger_entry(account_id)` — right now there isn't one, so it's a sequential scan. Step two is pushing the aggregation into SQL — a GROUP BY sum — and step three, when needed, the incremental aggregate balance."

Trap: Saying "I use a SQL SUM query". Wrong — and if they ask you to open `UserController.getMe`, the loop is right there. Precision builds trust; glossing destroys it.

Project evidence: `UserController.getMe` (loop over `ledgerEntryRepository.findByAccountId`), `AccountController.findBalance`. [CODE]

Q17. "Why is the ledger append-only? How does immutability help auditing?"

Priority: MUST KNOW

What interviewer is testing: Understanding of immutability as a design property, not a buzzword.

Ideal natural answer: "Because editability is the enemy of audit. If history can be changed, you can't distinguish a correction from a cover-up. So entries are insert-only — my code has no update or delete path for `LedgerEntry` at

all. Mistakes get fixed with reversing entries, so both the error and its correction remain visible. Immutability also pays at runtime: append-only rows are never read-modify-written, so concurrent transfers can't lose updates, and the reconciliation invariant stays checkable at any moment."

Deep explanation: Three concrete audit wins: complete forensics (who/what/when via account, reference_id, createdAt); replayability (sum the history and you get today's balance — the ledger explains itself); and tamper-evidence at the design level — anything other than clean paired inserts is immediately suspicious.

Follow-up: "Your app code doesn't update rows — but a DBA could UPDATE the table. Then what?"

Follow-up answer: "True — immutability in v1 is by convention, not enforcement. In production I'd REVOKE UPDATE and DELETE on the ledger table from the app role, add a trigger-based audit trail for any privileged changes, and hash-chain the rows so tampering leaves evidence. Convention protects against bugs; permissions protect against people."

Trap: "Immutability means encrypting the data." Different concept entirely. Also don't claim DB-level enforcement exists — it doesn't; say 'convention today, grants in production'.

Project evidence: LedgerEntryRepository — only save + findByAccountId; no modifying methods anywhere. [CODE]

Q18. "What happens in your code if the debit succeeds but the credit insert fails?"

Priority: MUST KNOW

What interviewer is testing: Whether atomicity is something you built or something you've only read about.

Ideal natural answer: "It can't half-happen. Both inserts are inside LedgerService.transfer, which is @Transactional — one database transaction. If the credit insert throws, Spring marks the transaction rollback-only and every statement in it — including the already-executed debit insert — is rolled back at the boundary. PostgreSQL simply discards the uncommitted work via its WAL recovery. So the caller gets an exception, the database state shows no trace of the attempt, and money was never half-moved. And when transfer is called from my fund flow, it joins the outer transaction with default REQUIRED propagation — so the debit, credit, status change and audit log all succeed or fail as one unit."

Deep explanation: This is ACID's 'A' doing real work. Worth naming the mechanics: Spring's proxy begins the tx, the JDBC connection executes both INSERTs on it, the RuntimeException triggers rollback, and uncommitted tuples are invisible to everyone (MVCC). The failure scenario interviewers fear — money deducted with no credit — is structurally impossible here.

Follow-up: "What if transfer wasn't transactional — walk the damage."

Follow-up answer: "The debit persists, the credit doesn't: the buyer's derived balance drops with no seller or escrow credit — money destroyed from the system's view. $\sum \text{debits} \neq \sum \text{credits}$, so reconciliation flags it, but the customer harm is done. That's exactly why every money path in the code is transactional."

Trap: "Hibernate retries the failed insert." No — it rolls back. Retries are a client concern, and a retry after rollback is safe precisely because nothing was committed.

Project evidence: LedgerService.transfer @Transactional; called within EscrowService.fundTransaction/confirmEscrow (also @Transactional). [CODE]

Q19. "How does the ledger help reconciliation? Give me a concrete query."

Priority: HIGH

What interviewer is testing: Whether "audit trail" is a real capability or a hand-wave.

Ideal natural answer: "Two checks. First, the conservation check — do the books balance at all: SELECT type, SUM(amount) FROM ledger_entry GROUP BY type; if those two numbers ever differ, something wrote one-sided entries, because my only write path creates pairs. Second, the escrow-specific proof — is the money I claim to be holding actually there: compute ESCROW_HOLDING's net as SUM of credits minus debits on that account, and compare it against SELECT COALESCE(SUM(amount),0) FROM escrow_transaction WHERE status IN ('FUNDED','SHIPPED','UNDER_REVIEW'). Those must be equal, because every funded-but-unreleased escrow contributed exactly one credit and no debit to the holding account. Any mismatch tells me exactly which invariant broke, and reference_id narrows it to the trade."

Deep explanation: Reconciliation = independent recomputation. The ledger lets you verify business claims (how much is held) against accounting facts (what the entries say) without trusting either. Because rows are immutable and paired, drift points to a code bug or external tampering — both worth alerting on.

Follow-up: "What can reconciliation NOT catch?"

Follow-up answer: "A perfectly paired malicious edit — a DBA adjusting both sides by the same amount stays 'balanced'. That's why reconciliation pairs with DB-level controls: revoke UPDATE/DELETE, audit triggers, hash-chaining. Detection and prevention are different layers."

Trap: Treating the ledger as the whole audit story. The honest framing: the ledger detects unbalanced corruption; it cannot detect balanced tampering without DB-level hardening.

Project evidence: LedgerEntry (type, amount, account FK, reference_id); statuses FUNDED/SHIPPED/UNDER_REVIEW are exactly the money-held states. [CODE] + [INFERENCE]

SECTION 4 — TRANSACTIONS + ACID

Q20. "What is a database transaction, in your project's terms?"

Priority: MUST KNOW

What interviewer is testing: Whether you can ground ACID theory in your own code instead of textbook recital.

Ideal natural answer: "A group of database operations that commits as one unit — all of it or none of it. The clearest example in my project is funding an escrow: inside one `@Transactional` method I insert a debit row, insert a credit row, update the escrow's status to FUNDED, and insert a status-log audit row. That's four statements across three tables that must live or die together. If any one fails, PostgreSQL rolls the whole thing back — so I can never end up with 'money moved but escrow still says CREATED' or the reverse."

Deep explanation: In Spring terms: the proxy intercepts the method, begins a transaction on the pooled connection, executes the body, commits on normal return, and rolls back on `RuntimeException`. The unit-of-work boundary sits at the service method — that placement is a deliberate architectural decision, not a default.

Follow-up: "How many transactions is a single fund request, really?"

Follow-up answer: "One. The outer `@Transactional` on `fundTransaction` opens it; `LedgerService.transfer` has `@Transactional` with default `REQUIRED` propagation, so it joins the existing transaction rather than opening a second — one commit at the outermost boundary."

Trap: "Every repository call is its own transaction." Technically each repository method would run in one if called standalone — but inside my transactional service methods they join the outer unit. Know the difference.

Project evidence: `EscrowService.fundTransaction` (Spring `@Transactional`) + `LedgerService.transfer` (jakarta `@Transactional`, `REQUIRED`). [CODE]

Q21. "Explain ACID using examples from RevertPay."

Priority: MUST KNOW

What interviewer is testing: Classic question; the differentiator is your-project mapping for every letter.

Ideal natural answer: "Atomicity: a release is a ledger debit, a ledger credit, a status update and an audit insert — if the status update fails, the ledger pair rolls back with it; there's no such thing as a half-release in my system. Consistency: invariants hold before and after — foreign keys, the unique idempotency key, `buyer ≠ seller`, and my domain's own law, $\Sigma \text{debits} = \Sigma \text{credits}$, which holds by construction. Isolation: PostgreSQL runs the default `READ COMMITTED`; concurrent transactions don't see each other's uncommitted rows, and my optimistic-lock version predicate re-checks the precondition at write time, so a stale reader can't commit over a newer state. Durability: once the release commits, it's in the write-ahead log — a crash right after the commit doesn't undo the seller's payout."

Deep explanation: The version-predicate point is the subtle one worth emphasizing: `READ COMMITTED` alone doesn't give you "check-then-act" safety — two transactions can both pass an in-memory check. The `WHERE id=? AND version=?` update turns the precondition into an atomic compare-and-swap at the storage layer. That's how A and I cooperate in the double-release scenario.

Follow-up: "Which isolation level are you on, and would you change it?"

Follow-up answer: "Whatever PostgreSQL defaults to — `READ COMMITTED`; nothing in my config overrides it. I wouldn't raise it: `SERIALIZABLE` would trade throughput and add serialization-failure noise for safety I already get from the version predicate and atomic pairs."

Trap: Claiming `SERIALIZABLE` or "strictest isolation everywhere". Saying the truth — default `RC` + version checks — shows you understand why, not just what.

Project evidence: `@Version` on `EscrowTransaction`; no isolation config anywhere (`application.properties` has none). [CODE] + [INFERENCE]

Q22. "Where exactly is `@Transactional` used in your code — and where is it missing?"

Priority: MUST KNOW

What interviewer is testing: This question separates people who placed transaction boundaries deliberately from people who sprinkled the annotation.

Ideal natural answer: "Used in exactly the three places where money or money-adjacent state moves: `LedgerService.transfer`, `EscrowService.fundTransaction`, and `EscrowService.confirmEscrow` — the release. Deliberately absent from pure reads. And honestly missing from several write paths I'd tighten: registration isn't atomic — user save, account creation and the seed transfer are separate transactions, so a crash mid-way can leave a user with no account. Similarly confirm, cancel, ship, `checkExpiry` and the dispute flow write the audit log and the escrow save as separate implicit transactions, so a crash between them can strand a log row whose state change never committed. My first refactor is putting `@Transactional` on those service methods — one annotation each, and those drift windows close."

Deep explanation: The missing-tx consequences are bounded — none involve money (fund/release are covered) — but they break the audit-chain invariant and can create broken onboarding states. Being able to enumerate your own gaps this precisely is the whole point of the question.

Follow-up: "Doesn't self-invocation break `@Transactional`?"

Follow-up answer: "It can — a `this.method()` call bypasses the Spring proxy, so a `@Transactional` method called from a sibling method in the same bean wouldn't get a new transaction. In my code, `transactionRequest` calls `this.checkExpiry` — harmless today only because `checkExpiry` isn't annotated. If I annotated it, I'd have to move it to another bean or use `TransactionTemplate`."

Trap: "Everything is transactional." The opposite is true and checkable in ten seconds of code reading. Precise honesty here is worth more than any perfect-sounding answer.

Project evidence: `@Transactional` at `EscrowService.java:192,252` and `LedgerService.java:37`; absent from `register/confirm/cancel/ship/checkExpiry/dispute`. [CODE]

Q23. "What happens if a release fails halfway through?"

Priority: MUST KNOW

What interviewer is testing: Atomicity under partial failure — and distinguishing DB failure from response failure.

Ideal natural answer: "Two different 'halfways'. If it fails mid-transaction — say the audit insert throws after the ledger pair succeeded — the whole transaction rolls back: both ledger rows vanish, the escrow stays SHIPPED, seller unpaid, and the client gets a 500. System state is exactly as if the request never happened, so a retry is completely safe. The nastier halfway is after commit but before the response reaches the client — network timeout. Then the release IS committed: seller got paid, status is RELEASED. The client thinks it failed and may retry — the retry hits the state machine, RELEASED is terminal, so no double payment; today it shows an error instead of 'already done', which is a UX gap, not a money gap."

Deep explanation: The lesson interviewers want: timeout $\neq$ failure. Once you can't distinguish "failed before commit" from "committed but response lost", you need idempotency or conflict-tolerant retry semantics. My state machine already gives the safety half; the graceful half is returning 409/200-with-state on replay instead of 500.

Follow-up: "Would a compensating transaction run automatically?"

Follow-up answer: "Nothing to compensate — atomicity means there's never a partial state needing reversal. Compensation matters in distributed, multi-resource flows; here everything is one local transaction."

Trap: "The system rolls back automatically after commit." No — after commit it's permanent (durability). Know which side of the commit boundary you're describing.

Project evidence: `confirmEscrow @Transactional`; terminal RELEASED in `isValid`. [CODE]

Q24. "Why must the money movement and the status update be atomic? What does V1 do vs what production should do?"

Priority: MUST KNOW

What interviewer is testing: Whether you understand why the transaction boundary is drawn exactly there.

Ideal natural answer: "Because those two writes are two views of one fact. If they commit separately, a crash between them gives you RELEASED with no payout — the seller is told they were paid and weren't — or FUNDED money moved while the escrow still says CREATED, inviting a second funding. Either breaks the trust the whole product is built on. In v1 both fund and release wrap the ledger pair, the status transition, and the audit log in one @Transactional method — one commit, one fact. Production additions on top: database-level CHECK constraints like amount > 0, transactional outbox events so notifications publish only from committed state, and the same atomicity extended to gateway-webhook-driven funding when real money rails arrive."

Deep explanation: The framing to hold: the transaction boundary should equal the business fact's boundary. A business fact here is "escrow #7 is funded, evidenced by ledger rows 101/102, logged with reason F". Anything less is a partial fact — the root cause of most real-world payment incidents.

Follow-up: "What breaks first if you moved the status update outside the transaction?"

Follow-up answer: "Crash timing becomes money loss: commit the ledger pair, crash before the status update — the escrow stays CREATED and can be funded again while money already moved. Double funding of a single escrow. That's the exact incident the current boundary prevents."

Trap: "It's atomic because databases are atomic." Databases are atomic within a transaction — someone had to draw the boundary correctly. Credit the design, not just the engine.

Project evidence: `fundTransaction/confirmEscrow` cover ledger + transition + log in one tx. [CODE]

SECTION 5 — CONCURRENCY

Q25. "Two requests hit 'Confirm Delivery' at the same moment. Walk me through exactly what happens."

Priority: MUST KNOW

What interviewer is testing: THE concurrency showcase of your project. They want the step-by-step, not the slogan.

Ideal natural answer: "Both requests load the same escrow — say SHIPPED, version 3. Both pass the ownership check and both pass the transition service, because validation runs against each thread's in-memory copy and neither has committed yet. Both insert their ledger pair and both attempt the update. Hibernate's update says: UPDATE escrow SET status='RELEASED', version=4 WHERE id=7 AND version=3. Request A's update matches one row and commits — money moves once. Request B's update matches zero rows, because A already bumped the version; Hibernate throws an optimistic lock failure, and because the whole method is @Transactional, B's two ledger inserts roll back with it. Final state: exactly one payout, exactly one release, and the loser gets an exception."

Deep explanation: Three mechanisms interlock, and the interviewer checks that you know all three are required: (1) the @Version column provides the compare-and-swap predicate at UPDATE time; (2) @Transactional ties the ledger inserts to that update so the loser's money rows die with it; (3) the state machine guarantees both attempts were individually legal — concurrency legality is layered on top of business legality, not mixed into it.

Follow-up: "What does the losing client actually see?"

Follow-up answer: "Today, an ugly 500 — the optimistic-lock exception has no handler; my ConcurrentModificationException class is still an empty stub. Correct behavior: catch it, re-read the escrow, and return 409 'already released' — or even 200 with the final state, since from the client's perspective the goal was achieved. That's on my fix list."

Trap: "Synchronization prevents it." Java synchronized only guards one JVM's threads — and doesn't cover the scheduler or a second app instance. The database row is the shared state; only database mechanisms arbitrate it.

Project evidence: @Version on EscrowTransaction; confirmEscrow @Transactional; exception/ConcurrentModificationException.java empty stub. [CODE] + [INFERENCE]

Q26. "What is optimistic locking, and why @Version?"

Priority: MUST KNOW

What interviewer is testing: Textbook concept + whether you can say why it fits this workload.

Ideal natural answer: "Optimistic locking assumes conflicts are rare: instead of locking a row while you work, you commit with a precondition — my escrow table has a version column, and every update carries WHERE version = the value I originally read. If someone committed in between, the version moved, my update matches nothing, and JPA throws ObjectOptimisticLockingFailureException. It fits escrow perfectly: a given escrow row is touched maybe five times in its whole life, by one buyer and one seller — contention is nearly zero, so paying lock-wait costs for pessimistic control would be waste. The version column is also the only thing standing between two concurrent releases and a double payout, which makes it the most safety-critical annotation in the codebase."

Deep explanation: Mechanically: JPA increments version on each flush; the WHERE-version predicate makes the UPDATE a database-level atomic compare-and-swap under READ COMMITTED — no locks held while the business logic runs. Failure mode is honest and immediate: you don't get silent corruption, you get an exception you must handle.

Follow-up: "Does the ledger table need @Version too?"

Follow-up answer: "No — versioning protects mutable rows from lost updates. Ledger rows are insert-only; there's nothing to overwrite, so there's nothing to lose. Versioning the ledger would mean admitting updates, which would break the immutability model."

Trap: Confusing @Version with @Transactional, or saying "it locks the row during the request". It locks nothing — it's a commit-time predicate. The row stays fully available to everyone until the UPDATE itself.

Project evidence: @Version private Long version in EscrowTransaction. [CODE]

Q27. "Why isn't your in-memory state validation enough to keep this safe?"

Priority: MUST KNOW

What interviewer is testing: The subtlest concurrency point in your project — business legality vs concurrency legality.

Ideal natural answer: "Because the validation runs on a snapshot. When two threads both read the escrow as SHIPPED, isValid happily returns true for both — each thread checks its own copy, and neither knows the other exists until commit time. So the switch statement answers 'is SHIPPED to RELEASED a legal move?' — yes — but it can't answer 'is it still legal by the time I commit?'. Only the database can answer that, which is exactly what the version predicate does at UPDATE time. I think of them as two layers: the state machine enforces business legality — no endpoint invents an illegal move; the version column enforces concurrency legality — stale movers lose at the finish line. Removing either layer breaks the system in a different way."

Deep explanation: This is check-then-act, the oldest race in the book: time-of-check to time-of-use. The general cure is making the check and the act atomic — here, folding the check into the UPDATE predicate. Naming "TOCTOU" and "compare-and-swap" shows you understand the pattern class, not just this instance.

Follow-up: "So is the transition service pointless for safety?"

Follow-up answer: "Not at all — it handles the 99% case that isn't concurrent: sequential illegal actions like funding twice, shipping a CANCELLED escrow, confirming an EXPIRED one. Those are rejected instantly with a clear domain exception. Concurrency safety is the version column's job; the machine's job is every non-racing error."

Trap: "My service is stateless so it's thread-safe." True but irrelevant — the escrow row is the shared mutable state, and it lives in the database. Thread-safety of your beans never addresses that.

Project evidence: EscrowTransitionService.isValid (in-memory, no locking); @Version predicate at UPDATE. [CODE] + [INFERENCE]

Q28. "Why not pessimistic locking — SELECT FOR UPDATE?"

Priority: HIGH

What interviewer is testing: Trade-off reasoning; whether you know optimistic isn't a dogma choice.

Ideal natural answer: "Pessimistic locking holds a row lock from SELECT until commit — guaranteed exclusivity, but at the cost of lock waits, lock management, and deadlock risk if locking order varies. For my workload it's overkill: each escrow row is a low-traffic resource touched a handful of times, so conflicts are rare and optimistic's fail-rarely-retry policy is cheaper and simpler. There's no @Lock anywhere and no FOR UPDATE in v1. Where I would reach for pessimism or its equivalents: enforcing a balance-floor — where the check must be atomic with the write — I'd use a conditional UPDATE like SET balance = balance - ? WHERE id = ? AND balance >= ?, or lock the aggregate row. And in any hot-row future — say a per-account aggregate under heavy write — pessimism earns its place."

Deep explanation: The decision rule: frequency of conflict decides. Rare conflicts → optimistic (pay only on the rare loss). Constant contention on one row → pessimistic (pay lock waits instead of endless retries). Deadlock avoidance under pessimism needs deterministic lock ordering — worth volunteering as a known cost.

Follow-up: "What happens if optimistic retries are naive?"

Follow-up answer: "Retrying inside the same transaction replays the same stale snapshot — you must retry the whole transaction: new transaction, fresh read, re-validate, backoff with jitter, capped attempts. And only for operations that are safe to re-run."

Trap: "Pessimistic is always safer." Safer against conflicts, yes — but it introduces deadlock and throughput failure modes optimistic doesn't have. 'Safer' depends on which failure you fear.

Project evidence: No @Lock/FOR UPDATE anywhere in the source. [CODE]

Q29. "The scheduler and a buyer action race on the same escrow — what happens?"

Priority: HIGH

What interviewer is testing: Cross-component concurrency: two entry points colliding on one row.

Ideal natural answer: "The collision is confirm-versus-expire on a PAYMENT_INITIATED escrow. Both load it; both pass their own validations — the transition service checks are in-memory, remember. Then the version column decides the winner. If the buyer's confirm commits first, the scheduler's update matches zero rows and throws an optimistic-lock exception inside the sweep — noisy log line, no damage; the escrow is CREATED and will fund normally. If the scheduler commits EXPIRED first, the buyer's confirm throws InvalidTransitionException — PAYMENT_INITIATED to CREATED is no longer legal from EXPIRED — and the user sees a 500. Crucially, no money is anywhere near this race — it happens entirely before funding — so the worst case is a confusing error, never a financial one."

Deep explanation: The two safety nets: the version column arbitrates the commit race; the state machine absorbs the "lost the race but arrived late" case as a clean domain rejection. Improvement worth naming: confirm should re-check confirmationDeadline before transitioning and fail fast with a 409 'payment window expired' — turning the race into deterministic behavior.

Follow-up: "Two app instances, both sweeping — disaster?"

Follow-up answer: "No — each sweep attempt re-validates through the transition service and the version column, so duplicate sweeps are wasteful and noisy but harmless. Production answer: ShedLock or a Quartz cluster so exactly one instance sweeps."

Trap: "The scheduler locks the rows it's processing." It doesn't — and saying so invites "show me the FOR UPDATE". The version column does the arbitrating.

Project evidence: EscrowScheduler → checkExpiry → transition; @Version arbitration; loop without per-item try/catch (one bad escrow aborts that tick). [CODE]

SECTION 6 — IDEMPOTENCY

Q30. "What is idempotency, and why do payments specifically demand it?"

Priority: MUST KNOW

What interviewer is testing: Vocabulary precision plus appreciation of why networks make this non-optional.

Ideal natural answer: "Idempotency means performing the same operation twice has the same effect as performing it once. Payments need it because retries are unavoidable: a request can timeout after the server committed, a connection can drop after the money moved, a user can double-click. From the client's perspective these are all 'did it work?' moments, and the natural response is retry. Without idempotency, every retry risks a second charge. The standard pattern is an idempotency key: the client generates one key per logical operation and sends the same key on every retry; the server recognizes it and replays the original outcome instead of executing again."

Deep explanation: The deep reason: distributed systems can't distinguish "failed" from "succeeded but response lost" without extra protocol. Idempotency keys turn ambiguous retries into safe retries — the server becomes the deduplicator of truth. GET/PUT/DELETE are naturally idempotent; POST is not, which is why POST endpoints in payments carry keys.

Follow-up: "Which of your endpoints are naturally idempotent?"

Follow-up answer: "The reads — buyer/seller lists, balances, ledger views. The mutators aren't naturally, which is exactly why initiate takes an Idempotency-Key and why the money endpoints lean on the state machine as a replay guard."

Trap: "Idempotent means it does nothing." That's nullipotent. Idempotent means doing it twice equals doing it once — a big difference with real money.

Project evidence: [GENERAL] concept, mapped to `EscrowController.initiate` key requirement. [CODE]

Q31. "What does V1 actually implement for idempotency — and what does it NOT?"

Priority: MUST KNOW

What interviewer is testing: Brutal honesty under pressure. This is the bluff-catcher of the whole interview.

Ideal natural answer: "Honestly — half of it. Implemented: the initiate endpoint requires an Idempotency-Key header and rejects missing or blank ones; the key is stored on the escrow row with a unique, not-null constraint, so the database itself guarantees one escrow per key. Not implemented: the replay path. I have `findByIdempotencyKey` declared in the repository — and never called — so a retried request with the same key doesn't get the original escrow back; it hits the unique constraint and gets a 500. The `requestHash` field that would detect 'same key, different payload' exists but is never written. And there's a client-side gap too: the frontend generates the key fresh on every button click, so a genuine network-retry from my own UI mints a new key and creates a duplicate escrow. Fund, ship and release take no keys at all — they rely on the state machine, which blocks double money movement but surfaces it as an error, not a friendly replay."

Deep explanation: The distinction the question hunts for: a unique constraint rejects duplicates (safe, hostile UX), while true idempotency replays the stored result (safe, graceful). V1 has the first, not the second. Every claim here is checkable in ten seconds of reading your code — which is exactly why full honesty is the only viable strategy.

Follow-up: "So is money at risk today?"

Follow-up answer: "No — the state machine plus optimistic locking make double payments impossible; a replayed fund or release throws instead of moving money twice. What's at risk is user experience: duplicate escrows from client retries and 500s instead of clean replays. That's the honest split: money safe, experience rough."

Trap: "My app is fully idempotent — I have a unique key constraint." The constraint only rejects; it never replays. Interviewers who know Stripe's model will dismantle that claim in one follow-up.

Project evidence: `EscrowController.initiate` (header required); `EscrowTransaction.idempotencyKey` unique+NOT NULL; `EscrowRepository.findByIdempotencyKey` never called; `requestHash` never written;

Q32. "What happens if the same payment request arrives twice — precisely?"

Priority: MUST KNOW

What interviewer is testing: Whether your honesty extends to concrete per-endpoint behavior.

Ideal natural answer: "Depends which request, and I'd split it that way. Initiate with the same key: the first insert succeeds; the second violates the unique constraint on idempotency_key and Spring turns it into a 500 — safe, one escrow exists, but the client should have gotten the original escrow back. Initiate with a fresh key — which is what my frontend actually sends on retry, since it regenerates per click: a second escrow row is created. That's the real duplicate-creation bug, and it lives client-side. Fund twice: the second call finds the escrow FUNDED, the transition service throws, no second debit — money safe, error shown. Release twice: identical story at SHIPPED. So: initiate can duplicate records, money endpoints cannot duplicate money."

Deep explanation: The three-tier answer (same-key initiate / fresh-key initiate / money endpoints) demonstrates you've traced every path rather than memorizing one line. The root cause hierarchy is worth stating: constraint safety is server-side and solid; key lifetime is client-side and broken; graceful replay is simply missing.

Follow-up: "How do you return the original escrow instead of a 500?"

Follow-up answer: "Lookup-first: findByIdempotencyKey before inserting; if found, return it with 200 or 409. And keep the constraint as the backstop — concurrent same-key inserts race past the lookup, so I'd catch the constraint violation and re-fetch by key, returning the winner's escrow."

Trap: Only answering for the happy "unique constraint saves me" path and forgetting the fresh-key duplicate — the one your own UI actually triggers. They may have your frontend code in view.

Project evidence: Constraint at EscrowTransaction; key generation in EscrowInitiate.jsx (Date.now()+Math.random); state-machine rejection in EscrowTransitionService. [CODE]

Q33. "How would you implement idempotency properly, end to end?"

Priority: HIGH

What interviewer is testing: Whether you can design the real mechanism, Stripe-style, given you didn't finish it.

Ideal natural answer: "Server side: a dedicated idempotency_keys table — the key, scoped per user as a composite unique, the request hash, a status of RECEIVED or DONE, and the stored response. The flow: insert the key row inside the same transaction as the business work; a concurrent duplicate blocks or fails on the primary-key insert and then polls or retries; on success the response snapshot is saved in that same commit; on replay, look up and return the stored response — never re-execute. If the hash differs for the same key, reject with 422, because the client is reusing a key for a different operation. Client side: mint the key once when the logical operation starts — a UUID held in form state — and reuse it across every retry until the operation reaches a final result. Then require keys on fund, ship and release too, not just initiate."

Deep explanation: The non-negotiable detail: the key insert and the business writes share one transaction — that closes the race window where two threads pass a pre-check simultaneously. The database's unique index is the only true serialization point; application-level checks alone are advisory. Key scoping per user prevents one user's keys colliding with another's.

Follow-up: "Where does the response storage live, and what's the cleanup story?"

Follow-up answer: "Response snapshot as JSON in the same table — simple, transactional, and replay is one indexed read. Nightly job purges keys older than 24–48 hours; uniqueness stays for the window that matters, storage stays bounded."

Trap: "Store the response in Redis after the transaction commits." That reopens a window — crash between commit and cache write means a replay re-executes. Same-transaction storage is the answer they're listening for.

Project evidence: Redesign target: EscrowService.transactionRequest + findByIdempotencyKey (declared, awaiting use). [CODE] + [GAP]

SECTION 7 — SPRING BOOT + JPA/HIBERNATE

Q34. "Why Spring Boot for this project?"

Priority: HIGH

What interviewer is testing: Framework choice reasoning — opinionated but honest.

Ideal natural answer: "Because a solo project still deserves a real architecture, and Boot gives the fastest honest path to one: dependency injection for wiring, embedded Tomcat so there's no separate server setup, and starters that pre-integrate the pieces a payments app genuinely needs — data-jpa, security, validation, web. Auto-configuration meant my security chain, JPA layer and connection pool worked from day one, so my time went into the escrow and ledger logic. I also made a deliberate choice to run the current stack — Spring Boot 4.1 on Java 25 — which forced me to read the new releases rather than copy older tutorials; even the starter names changed from what most blog posts show."

Deep explanation: The defensible core: Boot's value isn't convenience, it's sensible defaults with escape hatches — every default it made (Hikari pool, STATELESS-friendly security, Hibernate dialect) is one I could have named and overridden. Also worth one sentence: for something this small, plain Spring MVC would work, but I'd reimplement Boot's assembly by hand — starters are that assembly, packaged.

Follow-up: "What does a starter actually do for you?"

Follow-up answer: "Two things: pulls a curated, compatible set of dependencies — no version juggling — and triggers auto-configuration: beans get created conditionally based on what's on the classpath.spring-boot-starter-data-jpa on the classpath means a DataSource, an EntityManager and transaction management exist before I write a line."

Trap: "Because it's industry standard." Empty. Say what Boot did for this project: security chain, JPA wiring, scheduling, and one embeddable artifact.

Project evidence: pom.xml: parent 4.1.1, java.version 25, starters webmvc/data-jpa/security/validation. [CODE]

Q35. "Explain Controller vs Service vs Repository — using YOUR classes."

Priority: MUST KNOW

What interviewer is testing: Whether layering is real in your code or just slide vocabulary.

Ideal natural answer: "Controllers are transport-only: EscrowController reads the Authorization header, checks it says Bearer, pulls the path variable, and hands the raw token to the service — it holds zero business rules. The service layer is where everything decidable lives: EscrowService checks that the token's user is the escrow's buyer, asks EscrowTransitionService whether the state move is legal, tells LedgerService to move money, and orchestrates saves. Repositories are Spring Data interfaces — EscrowRepository with findByBuyerId, LedgerEntryRepository with findByAccountId — no SQL written by hand anywhere in v1. The rule I followed: a layer may only talk to the layer beneath it, and the transaction boundary sits at the service because that's where multiple repository calls compose into one business fact."

Deep explanation: Why not skip the service? Then controllers own rules and transactions — untestable without HTTP, duplicated across entry points (my scheduler reuses service logic precisely because it lives there), and HTTP-coupled. Why not fat repositories? Business logic in queries scatters invariants across SQL strings.

Follow-up: "Your controller passes a raw token string around — is that clean?"

Follow-up answer: "No, it's a known wart — the filter already authenticated; services re-validate defensively. The cleaner design: read the Authentication principal from the SecurityContext in the service, or a custom argument resolver, and drop token plumbing entirely. I kept v1 explicit so the security path is obvious when reading the code."

Trap: Describing layers generically. Name EscrowController → EscrowService → EscrowRepository and the reason each exists, or it sounds like you never opened your own code.

Project evidence: EscrowController, EscrowService, EscrowRepository, LedgerEntryRepository. [CODE]

Q36. "What is a Bean? How does Dependency Injection show up in your code?"

Priority: HIGH

What interviewer is testing: IoC fundamentals anchored to your classes.

Ideal natural answer: "A bean is any object the Spring container creates and wires instead of me — everything annotated `@Service`, `@Component`, `@RestController` in my app, plus `@Bean` methods like the `PasswordEncoder` and `SecurityFilterChain` in `SecurityConfig`, and the `CommandLineRunner` that seeds the `ESCROW_HOLDING` account at startup. Injection is constructor-based throughout — `EscrowService`'s constructor takes seven collaborators, Spring builds them in dependency order and hands them in. What I get: every dependency is explicit and final, testing means passing mocks to a constructor instead of fiddling with reflection, and there's exactly one instance of each service shared across requests."

Deep explanation: Constructor injection over field injection: immutability (deps can be final), fail-fast (the context refuses to start if a dependency is missing), and no reflection needed in tests. Component scanning finds beans because everything lives under the `@SpringBootApplication` package, `com.revertpay`.

Follow-up: "Seven dependencies in one constructor — is that a smell?"

Follow-up answer: "Yes, mildly — it says `EscrowService` orchestrates a lot. It's defensible for a coordinator of a bounded flow, but I'd extract money operations into a dedicated `EscrowMoneyService` and shrink the fan-in. Constructor injection at least makes the smell visible — field injection hides it."

Trap: "A bean is a class with getters and setters" — that's a `JavaBean`, a 90s convention. A Spring bean is a container-managed object. Different concepts that happen to share a word.

Project evidence: `EscrowService` constructor (7 deps), `SecurityConfig` `@Bean` methods, `RevertPayApplication` `CommandLineRunner`. [CODE]

Q37. "What does `@Transactional` actually do under the hood?"

Priority: MUST KNOW

What interviewer is testing: Beyond the annotation — proxy mechanics and rollback semantics.

Ideal natural answer: "Spring wraps the bean in a proxy. When another bean calls my method, the proxy opens a transaction on the pooled connection — or joins an existing one with default `REQUIRED` propagation — then invokes the real method. Return normally: commit. Throw a `RuntimeException`: mark rollback, and everything on that connection is undone. Two details people miss: it only works through the proxy, so a method calling a sibling via this bypasses the machinery entirely — the classic self-invocation trap; and rollback is runtime-exceptions-only by default — a checked exception would commit unless I set `rollbackFor`. In my code the mechanics matter concretely: `fundTransaction` and `transfer` compose into ONE transaction via propagation, so four statements across three tables commit or die together."

Deep explanation: Worth naming: the proxy is the reason `@Transactional` on a private method does nothing, and why the annotation must live where the caller crosses the boundary. My `LedgerService` uses the `jakarta.annotation.Transactional` variant while services use Spring's — both honor `REQUIRED` semantics here; in a review I'd standardize on Spring's for its richer options (isolation, propagation variants, `readOnly` hints).

Follow-up: "Does `@Transactional` lock rows or make things faster?"

Follow-up answer: "Neither. It defines an atomicity boundary — nothing more. Row protection in my design comes from the version predicate, and 'faster' is nonsense; transactions are about correctness under failure, at some throughput cost."

Trap: "It wraps the method in a database transaction" — right words, and then silence on proxy/self-invocation/rollback rules. That silence is where interviewers dig.

Project evidence: `LedgerService.transfer`, `EscrowService.fundTransaction`, `confirmEscrow`. [CODE]

Q38. "Entity vs DTO — and where does V1 get this wrong?"

Priority: MUST KNOW

What interviewer is testing: The honest one: can you name where entities leak into responses — including the security consequence.

Ideal natural answer: "Entities are database-mapped live objects — User, EscrowTransaction; DTOs are contract shapes — my RegisterRequest with @NotBlank and @Email, EscrowInitiateRequest with @Positive on amount, EscrowInitiateResponse for what initiate returns. DTOs decouple the API from the schema and give validation a home. And I'll volunteer where v1 violates its own rule: several endpoints return entities directly — the buyer and seller escrow lists serialize whole User objects, which means the password hash goes out in the JSON. That's the worst kind of leak because it's silent. The fix is mechanical: @JsonIgnore on the field as a band-aid, proper response DTOs as the real fix, and a contract test asserting the word 'password' never appears in a response body."

Deep explanation: Why entities-as-DTOs fail beyond the leak: over-fetching (clients receive fields they don't need), lazy-loading exceptions during serialization, tight coupling (renaming a column breaks clients), and security by omission (every new field auto-exposes). The password-hash leak is the demonstrable, severe one — knowing it before the interviewer does converts a vulnerability into a judgment display.

Follow-up: "Why is returning entities so tempting that codebases do it anyway?"

Follow-up answer: "Because it's free — Spring serializes whatever you return. Discipline costs a mapping layer. The compromise I'd ship next: Jackson @JsonIgnore on sensitive fields immediately, record-style response DTOs per endpoint after, generated from the controller signatures so drift is visible in review."

Trap: "My project uses DTOs everywhere." Directly false for the escrow lists — and it's the one place they'll check, because the master-gap list includes the hash leak.

Project evidence: `dto/*` with validation vs entity returns in `getBuyerTransactions/getSellerTransactions;` `User.password` without `@JsonIgnore`. [CODE] + [GAP]

Q39. "Explain persistence context, lazy vs eager, and the N+1 problem — in your listing endpoints."

Priority: MUST KNOW

What interviewer is testing: JPA internals with a live N+1 in your own code.

Ideal natural answer: "The persistence context is Hibernate's session-level workspace: one identity map where each row is at most one object, dirty-checked and flushed at commit — which is why my `escrowRepository.save` calls after mutating status are partly redundant; managed entities flush themselves. On loading: @ManyToOne associations default to EAGER, and my `EscrowTransaction` has two — buyer and seller. So `findByBuyerId`, which looks like one query, actually issues one query for the list plus two per escrow as associations materialize. That's textbook N+1: with 100 escrows, ~201 queries. It works, but it's the first performance fix I'd make: switch associations to LAZY, then fetch what each screen actually needs with an @EntityGraph or a DTO projection query — one joined query, exactly the columns the UI shows."

Deep explanation: Why nothing crashes today: because ManyToOne is eager by default, the associations are already loaded when Jackson serializes — no LazyInitializationException. Spring Boot's open-session-in-view also masks lazy issues for requests. Both masks disappear the moment someone flips fetch type without adding a fetch plan — which is precisely why the fix must pair LAZY with explicit fetching.

Follow-up: "How would you prove the N+1 is real?"

Follow-up answer: "show-sql is already true — one request to `/api/escrow/buyer` with a few escrows prints the tell-tale pattern: one select from `escrow_transaction`, then alternating selects on users. Count the statements, apply the EntityGraph, count again. I'd also add a test asserting query count stays at one."

Trap: "I use lazy loading everywhere." The code's defaults say otherwise — ManyToOne is EAGER unless declared otherwise, and yours aren't declared. Precision here = ownership.

Project evidence: `EscrowTransaction.buyer/seller (@ManyToOne, no fetch override);` `spring.jpa.show-sql=true`. [CODE] + [INFERENCE]

Q40. "Where does @Version fit into JPA — and what else does your JPA layer deliberately NOT use?"

Priority: HIGH

What interviewer is testing: JPA breadth and restraint — knowing what you skipped and why.

Ideal natural answer: "@Version is JPA's optimistic-locking hook: a numeric column JPA checks and increments on every update — it's what makes my two-concurrent-releases story safe. As for what I deliberately don't use: no handwritten JPQL or @Query anywhere — every repository method is a derived query like findBySellerId or findByIdempotencyKey, which is honest Spring Data at this scale; no @OneToMany collections — I intentionally query child-to-parent instead of letting an entity drag a list into memory; no cascades — nothing should disappear because its parent did, which in payments is exactly right. And every enum maps with EnumType.STRING, never ORDINAL — reordering an enum constant must never silently rewrite history. Trade-offs I accepted: IDENTITY id generation disables JDBC insert batching, and derived queries mean the day I need SELECT FOR UPDATE or a SUM, I add @Query — the layer grows when the need is real."

Deep explanation: Each restraint maps to a domain reason: no cascades ↔ immutability of money history; no @OneToMany ↔ no accidental full loads; STRING enums ↔ data survival across refactors. This is the answer pattern interviewers want: not "I know every feature" but "I know why each feature is or isn't here".

Follow-up: "When exactly would you finally write a @Query?"

Follow-up answer: "Three triggers: a locking read (FOR UPDATE for pessimistic enforcement), a server-side aggregation (balance as SUM instead of loading rows), and pagination with a deterministic sort for the listing endpoints. None of those needs exist today without breaking correctness — which is why the queries don't exist yet."

Trap: Listing every JPA feature you know. The question rewards selective knowledge — name the two you use (@Version, derived queries), the several you skip, and the domain reason for each.

Project evidence: @Version field; repositories with only derived methods; @Enumerated(EnumType.STRING) on all enums; zero @Query. [CODE]

SECTION 8 — SECURITY

Q41. "Authentication vs authorization — how does YOUR API do each?"

Priority: MUST KNOW

What interviewer is testing: Two concepts in one breath — the classic; precision on where each happens in your code.

Ideal natural answer: "Authentication is who are you — in my API, the JWT. Every request carries a Bearer token; my `JwtAuthenticationFilter` verifies the signature and expiry through `JwtService`, then loads the actual user row from the database, and stores an authenticated principal with a `ROLE_BUYER` or `ROLE_SELLER` authority in the `SecurityContext`. Authorization is what may you do — that happens on two levels in my code: coarse, URL-level rules in `SecurityConfig` where only register and login are public and everything else demands authentication; and fine-grained, object-level checks inside the services — funding compares the token's user id with the escrow's buyer id, shipping compares with the seller id, and disputes require the buyer. What I don't have is annotation-based method security — no `@PreAuthorize` yet; role rules live as service logic."

Deep explanation: The DB-load in the filter is a real design decision: authorization always sees the user's current role, so a token minted before a role change can't ride stale privileges — at the cost of one query per request. The trade's alternative (trust the token's role claim) is faster but freezes authority at issue time.

Follow-up: "Where exactly is the ownership check for funding — quote it?"

Follow-up answer: "In `fundTransaction`: extract user id from the token, load the escrow, then `if (!token_id.equals(transaction.getBuyer().getId())) throw` — object-level authorization: a valid token still only acts on escrows that belong to that identity."

Trap: "Spring Security handles authorization." It handles the URL rules; object-level checks are yours and live in services. Blurring them is how horizontal-privilege bugs happen — interviewers know it.

Project evidence: `JwtAuthenticationFilter`, `SecurityConfig.authorizeHttpRequests`, ownership compares in `EscrowService`. [CODE]

Q42. "What's inside your JWT, and how is it verified?"

Priority: MUST KNOW

What interviewer is testing: JWT anatomy from your generator, plus verification mechanics.

Ideal natural answer: "Three dot-separated base64url parts. Header: HS256. Payload: subject is the user's id as a string, plus email and role claims, issued-at, and expiry one hour out. Signature: HMAC-SHA256 over the first two parts using a secret from my properties file. Verification is two steps: `parseSignedClaims` re-computes the signature and throws on any tampering or structural problem, then I explicitly check expiration is in the future — any exception means the token is invalid, and my `isTokenValid` returns false. Identity flows from the subject claim only; the role claim inside the token is informational, because authorization always re-reads the role from the database."

Deep explanation: The signed-not-encrypted distinction is the classic probe: anyone can read the payload — which is why it carries only id, email, role, never secrets. Tampering demo: flip sub from 5 to 9 and the signature check fails immediately — that's the whole security model of JWT in one sentence. Honest gap to own: no issuer/audience claims and no jti, which limits revocation options later.

Follow-up: "Where do you store it client-side, and is that safe?"

Follow-up answer: "localStorage — which makes it immune to CSRF since nothing is auto-attached, but exposed to XSS: any script in the page can read it. Mitigations are Content-Security-Policy, short expiry — one hour here — and for a production version, httpOnly cookies with a CSRF token instead, accepting the CSRF work in exchange for XSS-proofing."

Trap: "JWTs are encrypted." They're signed. If you say encrypted, the very next question is "so what's inside?" — and you've contradicted yourself.

Project evidence: `JwtService.generateToken/extractClaims/isTokenValid`; secret via `@Value("${jwt.secret}")`. [CODE]

Q43. "Why BCrypt for password hashing?"

Priority: HIGH

What interviewer is testing: Crypto-adjacent judgment: hash vs encrypt, salt, cost factor.

Ideal natural answer: "Because password hashing needs to be slow, and BCrypt is deliberately slow with a tunable cost factor, plus a per-hash salt baked into the hash string itself — no separate salt storage. On registration, `passwordEncoder.encode` produces the hash we store; on login, `passwordEncoder.matches` re-hashes the attempt and compares against the stored hash in constant time, which avoids timing leakage. I chose it over SHA-256 because a fast general-purpose hash lets an attacker brute-force billions of guesses on stolen hashes; BCrypt's cost makes that expensive per guess, and the cost factor scales with hardware over time."

Deep explanation: The three pillars to name: one-way (never decryptable — 'encryption of passwords' is the giveaway mistake), salted (identical passwords produce different hashes, killing rainbow tables), adaptive (cost factor raises the bar as GPUs improve). Verification works without a salt column because the algorithm, cost and salt are all encoded in the stored hash string.

Follow-up: "Where exactly does your code encode and match?"

Follow-up answer: "AuthService: register calls `passwordEncoder.encode` before saving the user; verify calls `passwordEncoder.matches` on the login attempt against the stored hash, and both wrong-password and unknown-email return the same generic message — which also closes the user-enumeration side channel."

Trap: "We encrypt passwords with AES" or "we hash with MD5+salt". The first misunderstands one-wayness; the second picks a hash built for speed — exactly wrong for passwords.

Project evidence: `SecurityConfig.passwordEncoder() → BCryptPasswordEncoder; AuthService.register/verify.` [CODE]

Q44. "Why stateless authentication? What are the trade-offs?"

Priority: HIGH

What interviewer is testing: Architecture reasoning — statelessness isn't free; what did you trade?

Ideal natural answer: "My security config sets `SessionCreationPolicy.STATELESS` — no server-side session, no cookie. Every request re-presents its own proof, the JWT. The payoff is horizontal scaling: any instance can serve any request because nothing lives in server memory — which is also why the scheduler and multiple future instances stay simple. The costs are real and I can name them: no revocation — there's no logout that invalidates a token server-side; logout is just deleting it from `localStorage`, and a stolen token lives its full hour. And since the token sits in `localStorage`, XSS is the threat model instead of CSRF. For v1 the trade is right; for production I'd shorten the access token to minutes and add refresh-token rotation with a revocable server-side record."

Deep explanation: The subtle correctness point: my filter loads the user from the DB per request, so I'm not fully trusting the token — privileges stay fresh. That's a hybrid: stateless transport, server-verified substance. Also worth one line: CSRF protection is disabled legitimately because auth is header-borne — no ambient cookie a cross-site page could exploit; re-enable it if cookies ever carry the token.

Follow-up: "How would you implement real logout?"

Follow-up answer: "You can't un-issue a stateless JWT, so: short access-token TTL, a refresh token stored hashed server-side and rotated on use, and logout revokes the refresh record; optionally a jti denylist cache for immediate kill-switch on compromise."

Trap: "Stateless is more secure." Different threat surface, not more security: you trade CSRF exposure for XSS exposure and gain revocation headaches. Show you chose it for scale and simplicity, with eyes open.

Project evidence: `SessionCreationPolicy.STATELESS; csrf.disable();` 1-hour expiry in `JwtService`; `localStorage` in `App.jsx`. [CODE]

Q45. "What are the security gaps in V1? Rank them."

Priority: MUST KNOW

What interviewer is testing: The vulnerability sweep. Finding these before they do is the difference between owner and tourist.

Ideal natural answer: "Ranked by severity. One: the escrow list endpoints serialize full User entities, so counterparties' BCrypt password hashes ship in the JSON — no @JsonIgnore, no DTO. Two: the manual transfer endpoint, /api/auth/transfer, lets any authenticated user move money between any two accounts with no balance check — arbitrary money movement. Three: account endpoints expose any account's balance and full ledger to any logged-in user — no ownership filter. Four: anyone can self-register as ADMIN, since registration trusts the client's role choice. Five: database credentials and the JWT secret are committed in application.properties. Six: no rate limiting or lockout on login. Seven: no refresh/revocation — a stolen token works for an hour. Each has a one-line-to-one-day fix, and I'd start with the hash leak and the transfer guard."

Deep explanation: The meta-point: your security posture is judged less by having gaps than by ranking and remediation fluency. One and two are the critical pair — one is data exfiltration, two is money movement by an unauthorized party. Three follows because financial data is PII-adjacent. Volunteering them with fixes converts weaknesses into evidence of security review skill.

Follow-up: "Fix the top two — concretely."

Follow-up answer: "Hash leak: add response DTOs (or @JsonIgnore immediately) for every endpoint returning User, plus a test asserting 'password' never appears in any response body. Transfer: require account1 to be the caller's own account, resolve ownership from the token, enforce a non-negative balance inside the transaction, and return proper 4xx codes instead of 200-with-a-message-string."

Trap: "No known vulnerabilities." Everyone knows that's false the moment they read your list endpoints. The bluff here is uniquely easy to catch — and uniquely damaging.

Project evidence: Entity serialization in `getBuyerTransactions`; `LedgerController.transfer`; `AccountController` lookups; `RegisterRequest` role field; `application.properties` secrets. [CODE] + [GAP]

SECTION 9 — FAILURE / DESIGN ATTACK

Q46. "The buyer has insufficient balance and clicks Fund. What happens?"

Priority: MUST KNOW

What interviewer is testing: Whether you know your most dangerous gap — or will pretend it doesn't exist.

Ideal natural answer: "Nothing stops it — and that's my biggest known gap. V1 has no balance check on any money path: the fund endpoint debits the buyer and credits escrow regardless, so the buyer's derived balance simply goes negative and the escrow becomes FUNDED. There's an `InsufficientFundsException` class in the exception package — still an empty, unused stub. The honest flow is: 'that isn't implemented in my V1; the transfer just happens and the account goes negative. I'd implement it by computing the buyer's balance — ultimately from a maintained aggregate — inside the fund transaction, and throwing before any ledger write if it's short.' The design subtlety: a plain `SELECT`-then-check doesn't survive concurrency — two simultaneous funds both pass the check — so the real fix is atomic: a conditional update or a locked aggregate row."

Deep explanation: Why the naive check fails: under `READ COMMITTED`, two transactions can both read balance 600 and both fund 500 — both checks pass, both commit, balance -400. The check must be part of the write: `UPDATE ... WHERE balance >= ?`, or `SELECT FOR UPDATE` on the balance row. That's the answer's core — enforcement must be atomic with the movement, exactly like the version predicate is atomic with the status change.

Follow-up: "So could someone drain the platform this way?"

Follow-up answer: "Within v1 as deployed, a buyer could fund escrows they can't afford — that's why I call it the top gap. The conservation law still holds globally — Σ debits still equals Σ credits, the ledger isn't corrupt — but the policy is unenforced. Conservation is a property the design guarantees; solvency is a rule the code must add."

Trap: "It rejects with `InsufficientFundsException`." The class exists but is `EMPTY` and `UNTHROWN` — this is checkable in five seconds. Claiming enforcement that doesn't exist is the single worst bluff available in this codebase.

Project evidence: No balance comparison anywhere in `EscrowService/LedgerService`; `exception/InsufficientFundsException.java` empty stub. [CODE] + [GAP]

Q47. "Scheduler expires an escrow at the exact moment the buyer confirms. Outcome?"

Priority: MUST KNOW

What interviewer is testing: Race analysis with a specific winner matrix — no hand-waving.

Ideal natural answer: "Both load the escrow as `PAYMENT_INITIATED` — the scheduler from `findByStatus`, the buyer through `confirmTransaction` — and both validations pass in memory. Then the version column picks a winner. If the buyer commits `CREATED` first, the scheduler's update matches zero rows and dies with an optimistic-lock exception in that sweep iteration; escrow stays `CREATED`, funds normally, and the log noise is the only scar. If the scheduler commits `EXPIRED` first, the buyer's confirm throws `InvalidTransitionException` — `CREATED` is unreachable from `EXPIRED` — and the user sees a 500. Either way zero money is involved: this stage is entirely pre-funding. The improvement I'd make: confirm should re-read the `confirmationDeadline` and fail fast with a 409 'payment window expired' instead of racing the sweeper at all."

Deep explanation: The general lesson: races between an automated actor and a user actor resolve safely when (a) commit-time arbitration exists — the version predicate — and (b) late arrivals hit a domain rejection — the state machine — rather than corrupting state. Money-safety here is trivial because the boundary design keeps expiry strictly on the zero-money side of the lifecycle.

Follow-up: "Where does the buyer's 500 actually surface?"

Follow-up answer: "As a toast in the buyer UI saying confirmation failed — because the exception handler is empty, the user can't tell 'you were too slow' from 'the server broke'. The fix is the same as the deadline re-check: a deliberate 409 with a clear message, mapped in a `GlobalExceptionHandler`."

Trap: "The scheduler always wins because it runs every 5 seconds." Frequency doesn't grant priority — commit order does, and either side can win. Present the matrix, not a guess.

Project evidence: `EscrowScheduler + checkExpiry vs confirmTransaction; @Version` arbitration; no deadline re-check in `confirm`. [CODE]

Q48. "Can your system actually refund money today?"

Priority: MUST KNOW

What interviewer is testing: The namesake feature of "Revert"Pay — and whether you'll claim it works.

Ideal natural answer: "No — and I say that plainly. The refund exists as design scaffolding: the state machine models `UNDER_REVIEW` to `REFUNDED`, and the escrow row has `resolvedAt` and `resolution` columns, but both sit unused — no endpoint, no money movement, no admin flow. If you asked me to build it, it'd be a day: a refund endpoint gated to admins, valid only from `UNDER_REVIEW`, doing a reversing double-entry transfer — `ESCROW_HOLDING` debited, buyer's account credited, `reference_id` still the escrow id so the fund-and-refund pair tells the whole story — with the `REFUNDED` transition, the `resolution` fields, and the dispute status update all in one `@Transactional` block. Double-refund is already impossible because `REFUNDED` is terminal; the concurrent refund-versus-release race is already covered by the version column."

Deep explanation: Why this design is safe-by-construction: a refund is just another transfer — the ledger core absorbs it with no schema change, and the existing invariants (pairing, conservation, terminal-state uniqueness) all apply automatically. That's the payoff of building the money primitive correctly the first time: new products compose instead of patching.

Follow-up: "What would break first when you add it?"

Follow-up answer: "Authorization — there are no admin-only endpoints or provisioning yet, so the guard has to come first or anyone with a token could refund escrows. Second, the dispute lifecycle: disputes currently stop at `OPENED`; resolution should update the dispute row in the same transaction."

Trap: "Yes, refunds work — there's a `REFUNDED` state." A state is not a flow. One "show me the endpoint" later and the interview is over. Volunteer the gap, then sell the design.

Project evidence: `isValid` allows `UNDER_REVIEW`→`REFUNDED`; `resolvedAt/resolution` never set anywhere; `DisputeStatus` never leaves `OPENED`. [CODE] + [GAP]

Q49. "PostgreSQL goes down mid-flow. Walk me through every state your system can be in."

Priority: HIGH

What interviewer is testing: Failure-mode thinking: crash consistency, durability, and where truth lives.

Ideal natural answer: "First, requests start failing fast — connection acquisition from the pool times out, callers get 500s; nothing hangs forever. For anything in flight: an uncommitted transaction is rolled back by the database on recovery — WAL has no commit record for it, so PostgreSQL discards it; my rollback guarantee means no ledger pair, no status change, no audit row survives half-done. For everything already committed: it's durable — WAL-flushed, survives the crash; a paid seller stays paid. After the DB returns, my application is stateless toward the database — it holds nothing in memory that matters — so it just resumes. Clients retry, and the design makes retries safe: replayed funds or releases hit the state machine, not the ledger. What I don't yet have is operational recovery posture: no health endpoint surfacing DB status, no HA configuration — that's deployment work, not code work."

Deep explanation: The architecture insight to land: the database is the only source of truth; the app holds no state — that's precisely why recovery equals database recovery. The failure taxonomy has exactly three buckets — never started, committed, mid-flight — and atomicity plus durability collapse the middle one away.

Follow-up: "What are your RPO and RTO?"

Follow-up answer: "Honestly, undefined in v1 — it runs on a single managed Neon instance with `sslmode` require, no replica, no PITR configured. Production design: continuous backups with point-in-time recovery puts RPO at seconds, a warm standby puts RTO at minutes, and reconciliation runs before money operations unfreeze."

Trap: "Requests queue until the DB comes back." They don't — the pool's connection timeout fails them with errors. Queuing-forever behavior would be worse: clients retry on errors; they abandon on hangs.

Project evidence: App is stateless toward DB (JWT, no sessions); all truth in PostgreSQL; @Transactional atomicity bounds every failure. [CODE] + [INFERENCE]

Q50. "What would you change before calling this production-ready?"

Priority: MUST KNOW

What interviewer is testing: Engineering judgment at the finish line: triage, sequencing, and self-knowledge.

Ideal natural answer: "I'd sequence it as three sprints. Sprint one, stop the bleeding: proper response DTOs so password hashes stop serializing; ownership plus balance enforcement on the transfer and account endpoints; server-side role provisioning so ADMIN isn't self-service. Sprint two, close the correctness gaps: a `GlobalExceptionHandler` mapping my domain exceptions to 401/403/404/409 instead of blanket 500s; `@Transactional` on the remaining state-changing methods so audit rows can't strand; wire the idempotency replay and fix the client key lifetime; build the auto-release job against `autoReleaseAt`; implement the refund flow admin-gated. Sprint three, productionize: Flyway migrations replacing `ddl-auto`, the test suite — state-machine matrix first, then a two-thread release race test on Testcontainers Postgres — secrets out of git and rotated, rate limiting on auth, indexes on the foreign keys, structured logging replacing my `println`s. Then I'd call it production-ready — and I'd be able to prove each line of it."

Deep explanation: The ordering logic: security first because exploitability beats everything; then money-correctness ergonomics — the safe-but-ugly seams (500s, stranded logs, replay gaps); then operability. Naming a first test (the state-machine matrix, pure and exhaustive) and a first index (ledger `account_id`) shows the plan is concrete, not a wish list.

Follow-up: "What's the single biggest risk if it shipped as-is?"

Follow-up answer: "The transfer endpoint — an authenticated user can move arbitrary money between arbitrary accounts with no balance rule. Everything else is polish by comparison; that one is a live hole, and it's why it heads sprint one."

Trap: "It mostly just needs deployment polish." No — you'd be shipping a password-hash leak and an open money endpoint. The candid, sequenced answer is the senior answer; the minimizer's answer ends the round.

Project evidence: All ten items trace to verified source: empty `GlobalExceptionHandler`, non-tx mutators, uncalled `findByIdempotencyKey`, stub `EscrowAutoReleaseJob`, unused `resolvedAt`, commented-out tests, committed secrets, unindexed FKs. [CODE] + [GAP]

THE 15 QUESTIONS I MUST ANSWER WITHOUT THINKING

1. **What is RevertPay?** → Escrow payment platform; money held in ESCROW_HOLDING until delivery confirmation; double-entry ledger; state machine; Spring Boot + PostgreSQL + JWT. (Q1)
2. **Walk one full escrow.** → initiate (no money) → confirm → fund (buyer→escrow) → ship → confirm-delivery (escrow→seller); states PI→CREATED→FUNDED→SHIPPED→RELEASED. (Q3)
3. **Draw the state machine.** → PI→CREATED|CANCELLED|EXPIRED; CREATED→FUNDED; FUNDED→SHIPPED; SHIPPED→RELEASED|UNDER_REVIEW; UNDER_REVIEW→RELEASED|REFUNDED; four terminals. (Q7)
4. **Why escrow?** → Temporal payment risk; a hold state between commitment and settlement; provable holdings via ledger. (Q6)
5. **What is double-entry and why?** → Every movement = paired equal debit+credit in one tx; $\Sigma D = \Sigma C$ always; audit + reconciliation + no lost updates. (Q13)
6. **₹1000 walkthrough.** → Seed D:ESCROW/C:ACC5 5000; fund D:ACC5/C:ESCROW 1000 ref7; release D:ESCROW/C:ACC9 1000 ref7; escrow net = amount held mid-flight, zero after. (Q14)
7. **Debit OK, credit fails?** → @Transactional rolls back both — money never half-moves; retry safe. (Q18)
8. **Two simultaneous releases?** → Both read SHIPPED v3; both pass in-memory checks; UPDATE...WHERE version=3 lets exactly one commit; loser's ledger rows roll back in its tx. (Q25)
9. **Why @Version? Why not synchronized?** → DB-level compare-and-swap at commit; synchronized guards one JVM, not the row. (Q26, Q25)
10. **Is V1 idempotent?** → Honestly half: key required + unique at initiate; replay NOT wired (repo method uncalled); frontend regenerates key per click; money endpoints state-machine-guarded but 500 on replay. (Q31, Q32)
11. **Where is @Transactional and where is it missing?** → transfer, fund, release have it; register/confirm/cancel/ship/checkExpiry/dispute don't — known drift windows. (Q22)
12. **How does the API know it's you and you may act?** → JWT verified in filter (signature+exp), user reloaded from DB; URL rules + service-level ownership compares (token id vs escrow buyer/seller). (Q41)
13. **What's in the JWT?** → HS256; sub=userId, email, role, 1h exp; signed not encrypted; tamper breaks signature. (Q42)
14. **Insufficient balance on fund?** → Not implemented — account goes negative; InsufficientFundsException is an empty stub; fix = atomic balance gate (conditional UPDATE / locked aggregate). (Q46)
15. **Production-ready? What changes?** → Sprint 1: DTO/serialization + transfer guard + admin provisioning. Sprint 2: exception handler, @Transactional closure, idempotent replay, auto-release, refund. Sprint 3: Flyway, tests, secrets, rate limits, indexes. (Q50)

5-MINUTE REVERTPAY REVISION

Architecture (say the chain): React/Vite (fetch, Bearer JWT, localStorage) → JwtAuthenticationFilter (verify + DB user load + ROLE authority) → SecurityFilterChain (STATELESS, CSRF off, permit register/login only) → Controller (thin) → DTO @Valid → Service (ALL rules + tx boundaries) → Repository (derived queries) → Hibernate ddl-auto=update → PostgreSQL @ Neon. Scheduler: @Scheduled(5000ms) sweeps expiry.

Escrow states: PI →(confirm) CREATED →(fund, \$) FUNDED →(ship) SHIPPED →(confirm-delivery, \$) RELEASED; PI →cancel CANCELLED; PI →(40s) EXPIRED; SHIPPED →dispute UNDER_REVIEW → RELEASED|REFUNDED (unimplemented flow). Terminals: CANCELLED, EXPIRED, RELEASED, REFUNDED. Deadline = now+40s at initiate; autoReleaseAt = ship+5min (job = stub).

Ledger: append-only ledger_entry(account, DEBIT/CREDIT, BigDecimal amount, reference_id, createdAt). transfer() = debit+credit, one tx. Balance = $\Sigma \text{credits} - \Sigma \text{debits}$ derived in Java loop (/api/me, /balance). No balance column (commented out in Account.java). Seed: D:ESCROW/C:ACC{id} ₹5000 (treasury negative = mint). Reconciliation: $\Sigma D = \Sigma C$; ESCROW net = $\Sigma(\text{FUNDED, SHIPPED, UNDER_REVIEW amounts})$.

Transactions: @Transactional exactly on transfer, fund, release. One fact = one tx (ledger pair + status + log). REQUIRED propagation joins transfer into fund's tx. Missing on register/confirm/cancel/ship/dispute/checkExpiry → log/save drift risk. Self-invocation bypasses proxy (this.checkExpiry — harmless only because unannotated).

Concurrency: @Version on escrow row = UPDATE...WHERE version=?; 0 rows → ObjectOptimisticLockingFailureException → loser's tx (incl. its ledger rows) rolls back. In-memory isValid = business legality only; version = concurrency legality. No @Lock/FOR UPDATE. Scheduler races resolve via version; no money pre-funding.

Idempotency: Key required at initiate; unique+NOT NULL stored. NOT wired: replay lookup (findByIdempotencyKey unused), requestHash unused, frontend key per-click, no keys on money endpoints. Money safe via state machine; UX rough (500s). Proper fix: idempotency_keys table (key+user PK, hash, response snapshot) in same tx.

JWT: HS256, secret in properties (committed — rotate!). sub=userId; email, role claims; 1h exp. parseSignedClaims + explicit exp check. Role claim NOT used for authz — DB role read per request. localStorage; no refresh/revocation.

Main classes: RevertPayApplication (CommandLineRunner → ESCROW_HOLDING) · SecurityConfig (BCrypt, STATELESS, chain) · JwtService/JwtAuthenticationFilter · EscrowController/Service/Repository · EscrowTransitionService (isValid switch) + EscrowStatusLogService · LedgerService/LedgerEntry · AccountService (ESCROW_HOLDING, seeding) · DisputeService (OPENED only) · EscrowScheduler.

Major gaps (volunteer, ranked): ① password hashes serialize in escrow lists ② /api/auth/transfer: no ownership, no balance check ③ account endpoints readable by anyone ④ ADMIN self-registration ⑤ secrets committed ⑥ empty GlobalExceptionHandler → 500s ⑦ refund money flow absent (resolvedAt/resolution unused) ⑧ auto-release stub ⑨ no rate limiting ⑩ commented-out tests, no Flyway, no FK indexes.

10 DANGEROUS FOLLOW-UPS

1. **"Show me where money can be created from nothing."** → `/api/auth/transfer` credits any account with no balance rule; account goes negative [GAP]. (The ₹5000 seed is intentional minting via treasury debit — distinguish policy violation from the designed mint.)
2. **"Prove a double payout is impossible — without hand-waving @Version."** → `UPDATE...WHERE id=7 AND version=3` is a DB-level compare-and-swap under `READ COMMITTED`: the second commit's predicate matches 0 rows → `StaleObjectState` → its tx (ledger inserts included) rolls back. Atomicity ties the money to the predicate's fate.
3. **"Your password hashes leak in JSON. How did that happen architecturally?"** → Entities returned as response bodies + class-level `@Getter`; no DTO boundary on list endpoints. Fix: response DTOs + contract test asserting 'password' never in a body.
4. **"Why is an empty `GlobalExceptionHandler` a security issue, not just UX?"** → Raw exception text leaks internals (class names, constraint names); inconsistent statuses break client error handling; stack traces print to stdout. Map domain exceptions → 401/403/404/409 with generic bodies.
5. **"A DBA edits one ledger row. What detects it?"** → Nothing, today: immutability is convention, not enforcement. Fix: `REVOKE UPDATE/DELETE`, audit triggers, hash-chaining. Reconciliation catches unbalanced edits, not paired tamper.
6. **"Same idempotency key, two different payloads — what does V1 do?"** → Second insert fails on the unique constraint (if first still exists) → 500; requestHash that would 422-reject it is never written [GAP]. Proper: same-key+different-hash → 422.
7. **"Five browser tabs click Confirm simultaneously. Walk it."** → Five concurrent releases; exactly one commit survives (version); others die on optimistic-lock or illegal-transition → 500s today. Improvement: re-fetch state; return 'already released' as success.
8. **"Why is `auth.substring(7)` acceptable — and when isn't it?"** → It assumes exactly "Bearer " prefix; a malformed scheme slips through unauthenticated (later caught by authorization) rather than rejected as 401. Robust: parse the scheme explicitly, 401 on anything unexpected.
9. **"Someone transfers ₹0.005. What happens?"** → `BigDecimal/NUMERIC` stores it; the INR formatter displays ₹0.00 (display rounding, value intact). Policy gap: no precision/scale pinned, no minimum amount — fix with `scale=2 + CHECK (amount>0)`.
10. **"Convince me you own this codebase in one sentence."** → "I can name every gap in it, rank them by severity, tell you the exact file and line each lives in, and the design that keeps the money core safe despite them — because I read and wrote every line."

PERFECT 60-SECOND PROJECT EXPLANATION

"RevertPay is an escrow payment platform I built end to end with Spring Boot and PostgreSQL. The problem it solves is trust between strangers in a trade: the buyer pays into a system-controlled escrow account instead of directly to the seller, the seller ships knowing the money is locked, and the buyer's delivery confirmation releases it through a double-entry ledger transaction. Three design decisions carry the project. First, there's no balance column — balances are derived from an append-only ledger, so every rupee has a traceable history and money structurally can't vanish. Second, every escrow moves through a centralized state machine — initiated, confirmed, funded, shipped, released — where illegal transitions are rejected and every change is audit-logged with actor and reason. Third, the escrow row is guarded by JPA optimistic locking, so even two simultaneous releases can only pay the seller once — the losing transaction's ledger writes roll back with it. Auth is JWT with Spring Security, per-object ownership checks in the service layer, and BCrypt-hashed passwords. And I'm clear-eyed about v1: refunds don't move money yet, idempotent replay isn't wired, and balance checks are missing — I can walk you through exactly how I'd add each."